REIMBURSEMENT MANAGEMENT SYSTEM

Complete Technical Documentation for Hackathon Development

React + Vite	Express.js	MySQL	JWT + OAuth
Frontend	Backend	Database	Auth

Team Members
- Yashodip More (Team Leader)
- Tejas Patil
- Jaykumar Girase
- Komal Kumavat

Event Details
- Odoo x VIT Pune Hackathon 26
- Problem: Reimbursement Management
- Mentor: Haja Ram (ramh-odoo)
- Date: 29 March 2026

1. PROJECT OVERVIEW

1.1 Problem Statement

Companies struggle with manual expense reimbursement processes that are time-consuming, error-prone, and lack transparency. The core pain points are:

- No simple way to define approval flows based on thresholds
- No support for multi-level approvals
- No flexible conditional approval rules

1.2 Solution Summary

A full-stack web application with role-based access (Admin, Manager, Employee) that automates the entire expense reimbursement lifecycle — from submission to multi-level conditional approval — with real-time currency conversion and OCR receipt scanning.

1.3 Tech Stack

Layer	Technology
Frontend	React 18 + Vite, TailwindCSS, React Router v6, Axios, React Hook Form, Zustand
Backend	Node.js, Express.js, JWT, Passport.js (Google OAuth), Nodemailer, Multer (file upload), Tesseract.js (OCR)
Database	MySQL 8 with mysql2 driver, schema migrations via plain SQL scripts
Auth	JWT (access + refresh tokens), Google OAuth 2.0, bcrypt password hashing
External APIs	restcountries.com (country/currency list), exchangerate-api.com (currency conversion)
Dev Tools	ESLint, Prettier, Git, dotenv, nodemon, concurrently

2. SYSTEM ARCHITECTURE

2.1 Folder Structure

Backend (Express)

```
reimbursement-backend/
├── src/
│   ├── config/
│   │   ├── db.js              # MySQL connection pool
│   │   ├── passport.js        # Google OAuth strategy
│   │   └── nodemailer.js      # Mail transporter config
│   ├── middleware/
│   │   ├── auth.js            # verifyToken, requireRole
│   │   └── upload.js          # Multer config for receipts
│   ├── routes/
│   │   ├── auth.routes.js     # /api/auth/*
│   │   ├── user.routes.js     # /api/users/*
│   │   ├── expense.routes.js  # /api/expenses/*
│   │   ├── approval.routes.js # /api/approvals/*
│   │   └── currency.routes.js # /api/currency/*
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   ├── user.controller.js
│   │   ├── expense.controller.js
│   │   ├── approval.controller.js
│   │   └── currency.controller.js
│   ├── services/
│   │   ├── approval.service.js   # Core approval logic
│   │   ├── ocr.service.js        # Tesseract OCR
│   │   ├── currency.service.js   # Conversion logic
│   │   └── mail.service.js       # Email notifications
│   └── utils/
│       └── helpers.js
├── uploads/                   # Temp receipt storage
├── .env
├── package.json
└── index.js                   # Entry point
```

Frontend (React + Vite)

```
reimbursement-frontend/
├── src/
│   ├── api/
│   │   └── axios.js           # Axios instance + interceptors
│   ├── store/
│   │   └── authStore.js       # Zustand global auth state
│   ├── components/
│   │   ├── Layout/
│   │   │   ├── Navbar.jsx
│   │   │   ├── Sidebar.jsx
│   │   │   └── ProtectedRoute.jsx
│   │   ├── UI/
│   │   │   ├── Badge.jsx      # Status badges (Pending/Approved/Rejected)
│   │   │   ├── Modal.jsx
│   │   │   ├── Table.jsx
│   │   │   └── LoadingSpinner.jsx
│   │   └── Expense/
│   │       ├── ExpenseForm.jsx
```

```
│   │       ├── ExpenseList.jsx
│   │       └── ApprovalChain.jsx
│   ├── pages/
│   │   ├── auth/
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   └── OAuthCallback.jsx
│   │   ├── admin/
│   │   │   ├── Dashboard.jsx
│   │   │   ├── UserManagement.jsx
│   │   │   └── ApprovalRules.jsx
│   │   ├── manager/
│   │   │   ├── Dashboard.jsx
│   │   │   └── PendingApprovals.jsx
│   │   └── employee/
│   │       ├── Dashboard.jsx
│   │       ├── SubmitExpense.jsx
│   │       └── MyExpenses.jsx
│   ├── hooks/
│   │   ├── useAuth.js
│   │   └── useExpenses.js
│   ├── utils/
│   │   └── formatCurrency.js
│   ├── App.jsx                # Router setup
│   └── main.jsx
├── .env
├── vite.config.js
└── package.json
```

3. DATABASE DESIGN (MySQL)

3.1 All Tables

companies

```
CREATE TABLE companies (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(255) NOT NULL,
  country     VARCHAR(100) NOT NULL,
  currency    VARCHAR(10)  NOT NULL,  -- e.g. INR, USD
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

users

```
CREATE TABLE users (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  company_id    INT NOT NULL,
  name          VARCHAR(255) NOT NULL,
  email         VARCHAR(255) NOT NULL UNIQUE,
  password_hash VARCHAR(255),          -- NULL if Google OAuth
  role          ENUM('admin','manager','employee') NOT NULL DEFAULT 'employee',
  google_id     VARCHAR(255),          -- NULL if email/pass login
  manager_id    INT,                   -- FK to users.id (their direct manager)
  is_active     BOOLEAN DEFAULT TRUE,
  created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (company_id) REFERENCES companies(id),
  FOREIGN KEY (manager_id) REFERENCES users(id)
);
```

expenses

```
CREATE TABLE expenses (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  employee_id     INT NOT NULL,
  company_id      INT NOT NULL,
  title           VARCHAR(255) NOT NULL,
  amount          DECIMAL(12,2) NOT NULL,   -- in submitted_currency
  submitted_currency VARCHAR(10) NOT NULL,
  amount_in_company_currency DECIMAL(12,2), -- converted on submission
  category        ENUM('travel','food','accommodation','equipment','other') NOT
NULL,
  description     TEXT,
  expense_date    DATE NOT NULL,
  receipt_url     VARCHAR(500),             -- uploaded file path
  status          ENUM('pending','approved','rejected') DEFAULT 'pending',
  current_step    INT DEFAULT 1,            -- which approval step is active
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  FOREIGN KEY (employee_id) REFERENCES users(id),
  FOREIGN KEY (company_id) REFERENCES companies(id)
);
```

approval_flows (Admin configures this)

```
CREATE TABLE approval_flows (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  company_id      INT NOT NULL,
  name            VARCHAR(255) NOT NULL,    -- e.g. "Standard Flow"
  condition_type  ENUM('sequential','percentage','specific_user','hybrid') DEFAULT
'sequential',
  percentage      INT,                      -- e.g. 60 (for percentage rule)
  specific_user_id INT,                     -- e.g. CFO user id (for specific rule)
  is_active       BOOLEAN DEFAULT TRUE,
  FOREIGN KEY (company_id) REFERENCES companies(id),
  FOREIGN KEY (specific_user_id) REFERENCES users(id)
);
```

approval_steps (Steps within a flow)

```
CREATE TABLE approval_steps (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  flow_id         INT NOT NULL,
  step_number     INT NOT NULL,             -- 1, 2, 3...
  approver_id     INT NOT NULL,             -- user with manager/admin role
  label           VARCHAR(100),            -- e.g. "Manager", "Finance", "Director"
  FOREIGN KEY (flow_id) REFERENCES approval_flows(id),
  FOREIGN KEY (approver_id) REFERENCES users(id)
);
```

expense_approvals (Tracks each approval action)

```
CREATE TABLE expense_approvals (
  id              INT AUTO_INCREMENT PRIMARY KEY,
  expense_id      INT NOT NULL,
  approver_id     INT NOT NULL,
  step_number     INT NOT NULL,
  action          ENUM('pending','approved','rejected') DEFAULT 'pending',
  comment         TEXT,
  acted_at        TIMESTAMP,
  created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (expense_id) REFERENCES expenses(id),
  FOREIGN KEY (approver_id) REFERENCES users(id)
);
```

refresh_tokens

```
CREATE TABLE refresh_tokens (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT NOT NULL,
  token       VARCHAR(512) NOT NULL,
  expires_at  TIMESTAMP NOT NULL,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id)
);
```

IMPORTANT: Always run DB migrations in this order — companies → users → expenses → approval_flows → approval_steps → expense_approvals → refresh_tokens

4. AUTHENTICATION & AUTHORIZATION

4.1 Auth Strategy Overview

Method	How It Works
Email + Password	bcrypt hash stored in DB. On login → issue JWT access token (15 min) + refresh token (7 days) in HttpOnly cookie
Google OAuth 2.0	Passport.js GoogleStrategy. On success → find or create user → issue same JWT + refresh token
Token Refresh	POST /api/auth/refresh — reads HttpOnly cookie, validates refresh token from DB, issues new access token
Logout	DELETE /api/auth/logout — deletes refresh token from DB + clears cookie

4.2 .env Variables Required

```
# Server
PORT=5000
NODE_ENV=development

# Database
DB_HOST=localhost
DB_PORT=3306
DB_NAME=reimbursement_db
DB_USER=root
DB_PASS=yourpassword

# JWT
JWT_SECRET=your_super_secret_jwt_key_here
JWT_EXPIRES_IN=15m
REFRESH_TOKEN_SECRET=another_secret_for_refresh
REFRESH_TOKEN_EXPIRES_IN=7d

# Google OAuth
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
GOOGLE_CALLBACK_URL=http://localhost:5000/api/auth/google/callback

# Nodemailer (Gmail SMTP)
MAIL_USER=yourteamemail@gmail.com
MAIL_PASS=your_gmail_app_password   # Use Gmail App Password, NOT account password
MAIL_FROM="Reimbursement System <yourteamemail@gmail.com>"

# Frontend URL (for CORS + OAuth redirect)
CLIENT_URL=http://localhost:5173

# Exchange Rate API
EXCHANGE_RATE_BASE_URL=https://api.exchangerate-api.com/v4/latest
```

4.3 JWT Middleware (auth.js)

Every protected route passes through verifyToken middleware. The middleware:

- Reads Authorization: Bearer <token> header
- Verifies JWT signature and expiry
- Attaches decoded user (id, role, company_id) to req.user
- Returns 401 if token missing/invalid, 403 if role insufficient

4.4 Role Authorization

requireRole middleware takes one or more roles as arguments:

```
// Usage in routes
router.get('/admin-only', verifyToken, requireRole('admin'), controller);
router.get('/managers-and-admins', verifyToken, requireRole('admin', 'manager'),
controller);

// middleware/auth.js
const requireRole = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Access denied' });
  }
  next();
};
```

4.5 First Signup Flow

When the very first user signs up (no company exists yet):

1. Accept: name, email, password, companyName, country
2. Detect country currency via restcountries API
3. Create company record with detected currency
4. Create user with role = 'admin' (first user always becomes admin)
5. Send welcome email via Nodemailer
6. Return JWT tokens

Google OAuth users who sign up first also become admin. Subsequent users created by admin will have role = 'employee' by default until changed.

5. API ENDPOINTS

Base URL: http://localhost:5000/api | All protected routes need: Authorization: Bearer <token>

5.1 Auth Routes (/api/auth)

Method	Endpoint	Role	Description
POST	/auth/register	Public	First signup — creates company + admin user
POST	/auth/login	Public	Email + password login → returns access token + sets refresh cookie
POST	/auth/refresh	Public	Refresh access token using HttpOnly refresh cookie
DELETE	/auth/logout	Auth	Invalidate refresh token, clear cookie
GET	/auth/google	Public	Initiates Google OAuth flow (redirect to Google)
GET	/auth/google/callback	Public	Google OAuth callback — creates/finds user, returns tokens
GET	/auth/me	Auth	Get currently logged in user info

5.2 User Routes (/api/users)

Method	Endpoint	Role	Description
GET	/users	Admin	Get all users in company
POST	/users	Admin	Create employee or manager account
GET	/users/:id	Admin	Get single user details
PUT	/users/:id	Admin	Update user info (name, role, manager_id)
PUT	/users/:id/role	Admin	Change user role (employee ↔ manager)
PUT	/users/:id/manager	Admin	Assign manager relationship
DELETE	/users/:id	Admin	Deactivate user (soft delete)
GET	/users/me/team	Manager	Get all employees under this manager

5.3 Expense Routes (/api/expenses)

Method	Endpoint	Role	Description
POST	/expenses	Employee	Submit new expense claim
GET	/expenses/mine	Employee	Get own expense history with status
GET	/expenses/:id	Auth	Get single expense detail + approval trail

Method	Endpoint	Role	Description
GET	/expenses	Admin	Get all expenses in company (with filters)
GET	/expenses/team	Manager	Get all expenses from team members
GET	/expenses/pending	Manager/Admin	Get expenses waiting for my approval
POST	/expenses/:id/ocr	Employee	Upload receipt image → OCR → return extracted fields
DELETE	/expenses/:id	Employee	Cancel expense (only if still pending step 1)

5.4 Approval Routes (/api/approvals)

Method	Endpoint	Role	Description
POST	/approvals/:expenseId/action	Manager/Admin	Approve or reject an expense at current step
GET	/approvals/:expenseId/trail	Auth	Full approval history for an expense
POST	/approvals/flows	Admin	Create new approval flow
GET	/approvals/flows	Admin	List all approval flows in company
PUT	/approvals/flows/:id	Admin	Update approval flow (steps, condition type)
DELETE	/approvals/flows/:id	Admin	Delete approval flow
POST	/approvals/override/:expenseId	Admin	Admin override — force approve/reject any expense

5.5 Currency Routes (/api/currency)

Method	Endpoint	Role	Description
GET	/currency/countries	Public	List all countries with currencies (from restcountries API)
GET	/currency/convert	Auth	Convert amount between currencies
GET	/currency/rates/:base	Auth	Get latest exchange rates for base currency

6. CORE BUSINESS LOGIC

6.1 Expense Submission Flow

7. Employee fills form: title, amount, currency, category, description, date, optional receipt
8. Frontend calls GET /api/currency/rates/{submitted_currency} to show converted amount live
9. On submit → POST /api/expenses
10. Backend: convert amount to company currency and store both values
11. Backend: if employee has a manager AND manager's 'is manager approver' concept is met → set current_step = 1 (manager step first)
12. Backend: create expense_approvals row for step 1 approver with action = 'pending'
13. Send email notification to step 1 approver via Nodemailer

6.2 Multi-Step Sequential Approval

Each flow has ordered steps. An expense moves forward only when current step approver acts:

```
APPROVAL LOGIC (approval.service.js):

ON APPROVE (step N):
  1. Mark expense_approvals[step N] = 'approved'
  2. Check if more steps remain:
     → YES: increment expenses.current_step, create next expense_approvals row,
send email to next approver
     → NO: mark expenses.status = 'approved', send email to employee

ON REJECT (any step):
  1. Mark expense_approvals[step N] = 'rejected', save comment
  2. Mark expenses.status = 'rejected'
  3. Send rejection email to employee with comment
```

6.3 Conditional Approval Rules

Rule Type	Logic
sequential	Default. Each approver must approve in order before moving to next step
percentage	If >= X% of all assigned approvers have approved → auto-approve expense
specific_user	If the specific designated user (e.g. CFO) approves at any step → auto-approve entire expense
hybrid	Either the percentage threshold is met OR the specific user approved → auto-approve

For hybrid and percentage rules, all approvers are notified simultaneously. The service checks after each approval action whether the condition is met.

6.4 OCR Service

- Employee uploads receipt image (JPG/PNG/PDF)
- Backend saves file via Multer to /uploads/
- Tesseract.js runs OCR on the image
- A regex-based parser extracts: amount, date, vendor name, line items
- Returns extracted fields as JSON to frontend
- Employee reviews, edits if needed, then submits expense

6.5 Email Notifications (Nodemailer)

Trigger	Recipient
Expense submitted	Step 1 approver — 'You have a new expense to review'
Step approved	Next step approver — 'An expense is waiting for your approval'
All steps approved	Employee — 'Your expense has been approved'
Any rejection	Employee — 'Your expense was rejected' + comment
New user created by admin	New user — welcome email with login credentials

Use Gmail App Password (not account password). Enable 2FA on Gmail first, then generate App Password at myaccount.google.com/apppasswords

7. FRONTEND PAGES & UI

7.1 Route Structure (React Router v6)

```
/ (root)
├── /login                      # Public
├── /register                   # Public (first company signup)
├── /auth/callback              # OAuth callback handler
├── /app                        # Protected (ProtectedRoute wrapper)
│   ├── /app/dashboard          # Role-based: shows different dashboard
│   ├── /app/expenses
│   │   ├── /new                # Submit expense form
│   │   ├── /mine               # My expense history (employee)
│   │   └── /:id                # Expense detail with approval trail
│   ├── /app/approvals          # Pending approvals (manager/admin)
│   ├── /app/users              # User management (admin only)
│   └── /app/settings
│       └── /approval-rules     # Approval flow config (admin only)
```

7.2 Page-by-Page UI Description

Login Page

- Two options: Email/Password form + 'Login with Google' button
- Email/password fields with validation (required, email format, min 6 chars password)
- On success: store access token in memory (Zustand), navigate based on role
- Show loading spinner during API call
- Show error toast on invalid credentials

Register Page (First Signup — Admin)

- Fields: Company Name, Your Name, Email, Password, Country (dropdown from restcountries API)
- Country selection auto-shows detected currency
- Tooltip: 'This will be your company's default currency'
- On submit: create company + admin user + redirect to dashboard

Dashboard (Role-Aware)

- Admin: total expenses, pending count, approved amount this month, top spenders chart
- Manager: my team's pending expenses count, quick approve buttons, recent activity
- Employee: my total submitted, approved, pending, rejected counts + quick submit button

Submit Expense Form (Employee)

- Fields: Title, Amount, Currency (dropdown), Category, Description, Expense Date, Receipt Upload
- Live currency conversion: as user types amount, show converted amount in company currency
- OCR button: 'Scan Receipt' → opens file picker → uploads → auto-fills form fields
- Receipt preview thumbnail if image uploaded
- Validation: all fields required except description and receipt

- Submit button disabled while loading

My Expenses (Employee)

- Table: Title, Amount, Category, Date, Status badge (color-coded), Action
- Status badges: Pending (yellow), Approved (green), Rejected (red)
- Click row → Expense Detail page with full approval chain
- Filter by status and date range

Expense Detail Page

- Top: expense info card (all fields)
- Middle: Approval Chain visualizer — step 1, step 2, step 3 with status icons and timestamps
- Bottom: Timeline of actions with comments
- If rejected: show rejection reason prominently

Pending Approvals (Manager/Admin)

- Table of expenses waiting for this user's action
- Shows: Employee name, Title, Amount (in company currency), Category, Date, Days waiting
- Each row has Approve and Reject buttons
- Reject opens modal with required comment text area
- Admin sees all pending expenses company-wide; manager sees only their assigned ones

User Management (Admin)

- Table of all users: Name, Email, Role, Manager, Status
- Create User button → modal form: name, email, password (auto-generated or set), role
- Edit user → change role, assign manager from dropdown (only users with manager role shown)
- Toggle active/inactive status

Approval Rules (Admin)

- List of approval flows with step count and condition type
- Create Flow: name it, add steps (drag to reorder), set approver for each step
- Condition type toggle: Sequential / Percentage (input %) / Specific User (dropdown) / Hybrid
- Mark a flow as 'Active' — only one active flow per company

8. STATE MANAGEMENT & API INTEGRATION

8.1 Zustand Auth Store

```
// store/authStore.js
import { create } from 'zustand';

const useAuthStore = create((set) => ({
  user: null,              // { id, name, email, role, company_id, currency }
  accessToken: null,
  isAuthenticated: false,

  setAuth: (user, token) => set({ user, accessToken: token, isAuthenticated:
true }),
  logout: () => set({ user: null, accessToken: null, isAuthenticated: false }),
  updateToken: (token) => set({ accessToken: token }),
}));
```

8.2 Axios Instance with Interceptors

```
// api/axios.js
import axios from 'axios';
import useAuthStore from '../store/authStore';

const api = axios.create({ baseURL: import.meta.env.VITE_API_URL, withCredentials:
true });

// Attach access token to every request
api.interceptors.request.use((config) => {
  const token = useAuthStore.getState().accessToken;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Auto-refresh on 401
api.interceptors.response.use(
  res => res,
  async (error) => {
    if (error.response?.status === 401 && !error.config._retry) {
      error.config._retry = true;
      const res = await axios.post('/api/auth/refresh', {}, { withCredentials: true
});
      useAuthStore.getState().updateToken(res.data.accessToken);
      return api(error.config);
    }
    return Promise.reject(error);
  }
);
```

8.3 Protected Route Component

```
// components/Layout/ProtectedRoute.jsx
import { Navigate } from 'react-router-dom';
```

```
import useAuthStore from '../../store/authStore';

export default function ProtectedRoute({ children, roles }) {
  const { isAuthenticated, user } = useAuthStore();
  if (!isAuthenticated) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/app/dashboard"
replace />;
  return children;
}

// Usage in App.jsx:
<Route path="/app/users" element={
  <ProtectedRoute roles={['admin']}>
    <UserManagement />
  </ProtectedRoute>
} />
```

9. SETUP & RUNNING LOCALLY

9.1 Backend Setup

14. git clone the repo, cd into backend folder
15. npm install
16. Create MySQL database: CREATE DATABASE reimbursement_db;
17. Run SQL scripts in /src/config/schema.sql to create all tables
18. Copy .env.example to .env and fill all variables
19. npm run dev (uses nodemon)
20. Backend runs at http://localhost:5000

9.2 Frontend Setup

21. cd into frontend folder
22. npm install
23. Create .env file: VITE_API_URL=http://localhost:5000/api
24. npm run dev
25. Frontend runs at http://localhost:5173

9.3 Google OAuth Setup

26. Go to console.cloud.google.com → Create new project
27. APIs & Services → Credentials → Create OAuth 2.0 Client ID
28. Authorized redirect URI: http://localhost:5000/api/auth/google/callback
29. Copy Client ID and Secret to .env

9.4 Git Workflow (IMPORTANT for Hackathon)

All 4 team members must commit code. Evaluators will check git log. Assign features so everyone pushes at least 3-4 meaningful commits.

Member	Ownership Area
Yashodip (Leader)	Repo setup, Auth system (JWT + OAuth), deployment, integration
Tejas	Expense submission, OCR service, file upload, currency conversion
Jaykumar	Approval workflow engine, approval rules, email notifications
Komal	Frontend UI — all pages, components, responsive design, Zustand store

```
# Git workflow
git checkout -b feature/your-feature-name   # always work on branches
git add .
git commit -m "feat: describe what you did"
```

```
git push origin feature/your-feature-name
# Then create PR → team leader reviews → merge to main
```

10. SUBMISSION CHECKLIST

Complete these in order. Don't skip steps. Check each one off before moving to next.

Timeline Checklist (29 March 2026)

Time	Action
9:00 AM	Coding starts. Initialize repos (backend + frontend). Push initial commit. Submit GitHub repo link on hackathon portal.
9:30 AM	Set up DB schema. Test DB connection. Set up Express server with health check route.
10:00 AM	Add mentor ramh-odoo as collaborator on GitHub repo. Verify they can see the code.
10:00 - 2:00 PM	Core coding sprint. Auth + Expense submission + Approval workflow.
2:00 - 4:00 PM	Frontend pages + UI polish + integrate APIs.
4:00 - 5:00 PM	Testing, bug fixes, README update, final commits from all members.
5:00 PM	Coding ends. Final push to main.
5:30 PM	Submit video demo link on portal.

Must-Have Features Checklist

- Authentication — email/password + Google OAuth working
- First signup creates company + admin automatically
- Admin can create employees and managers
- Admin can assign manager relationships
- Employee can submit expense with currency (different from company currency)
- Expense amount auto-converted to company currency on submission
- At least basic 2-step sequential approval working
- Manager can approve/reject with comment
- Employee sees their expense history with status
- Responsive UI with consistent color scheme
- Input validation on all forms
- All 4 members have commits in git log
- Mentor added as collaborator

Nice-to-Have Features (Do if time allows)

- Conditional approval rules (percentage / specific user / hybrid)
- OCR receipt scanning with Tesseract.js
- Email notifications via Nodemailer
- Admin override approval
- Dashboard charts with recharts or chart.js

- Expense filtering and search

All the best team! Ship it. 🚀

Odoo x VIT Pune Hackathon 26 | Reimbursement Management